



TRABALHO PRÁTICO – META I

DEIS – Inteligência Computacional – 2025/2026

Lecionado por:

Carlos Manuel Jorge da Silva Pereira – cpereira@isec.pt

Realizado por:

Pedro Saraiva – a2023146226@isec.pt
Daniel Soares – a2023144661@isec.pt

Índice

1. Introdução	2
2. Descrição do caso de estudo e objetivos do problema	2
3. Descrição do modelo e implementação dos algoritmos	3
3.1. Pré-processamento e Normalização das imagens.....	3
3.2. Modelo Base - Arquitetura CNN.....	4
3.3. Modelo alternativo – Arquitetura MLP.....	4
3.4. Configuração do Ambiente e Bibliotecas Utilizadas.....	5
4. Análise de resultados.....	6
4.1. Análise do Modelo Base (CNN)	6
4.2. Análise do Modelo Alternativo (MLP).....	7
4.3. Comparação entre MLP e CNN	8
5. Conclusões	9
6. Referências.....	9

1. Introdução

O presente projeto insere-se no âmbito da unidade curricular de Inteligência Computacional e tem como principal objetivo desenvolver um classificador de imagens capaz de identificar o tipo de carroçaria de um automóvel, recorrendo a redes neurais artificiais.

O projeto está dividido em três metas, sendo esta Meta I dedicada à análise do problema, à construção do dataset e ao desenvolvimento de um modelo base funcional. Nesta fase, o nosso grupo foca-se também em comparar as duas abordagens: MLP (Multilayer Perceptron) e CNN (Convolutional Neural Network) com o objetivo de justificar a escolha da rede neuronal mais adequada para o projeto.

2. Descrição do caso de estudo e objetivos do problema

O problema em estudo consiste na classificação automática de imagens de veículos em diferentes tipos de carroçaria. O sistema deve, a partir de uma imagem, ser capaz de identificar corretamente a categoria correspondente, distinguindo entre cinco classes:

A motivação deste trabalho passa por demonstrar a capacidade das redes neurais em aprender padrões visuais complexos, como formas, texturas e contornos característicos de cada tipo de veículo, sem necessidade de extração manual de características.



Entre as potenciais **aplicações reais** destacam-se:

- **Gestão de estacionamento inteligentes**, otimizando a alocação de vagas;
- **Seguros automóveis**, para identificação automática de veículos em fotografias enviadas por clientes;
- **Portagens eletrónicas**, para determinação automática da categoria do veículo;
- **Controlo de acessos**, como no caso do campus do ISEC, onde câmaras podem limitar o número de veículos de um determinado tipo.

O dataset utilizado foi baseado no conjunto de dados público “**Car Body Types Images Dataset**”, disponível na plataforma [Kaggle](#).

Contudo, este dataset foi modificado e expandido, de modo a garantir um número equilibrado de imagens e a conformidade com os requisitos mínimos definidos pela unidade curricular – 10.000 imagens no mínimo (dataset “**Stanford Car Body Type Data**”, também disponível na plataforma [Kaggle](#)).

O dataset final encontra-se dividido em três subconjuntos/subpastas:

Conjunto	Nº total de imagens	Distribuição por classe
Treino	8.000	Convertible: 1.759; Coupé: 1.594; Hatchback: 1.130; Sedan: 1.758; SUV: 1.759
Validação	1.000	200 por classe
Teste	1.000	200 por classe

Esta divisão segue a proporção de **80% treino / 10% validação / 10% teste**, com um balanceamento adequado nos conjuntos de validação e teste.

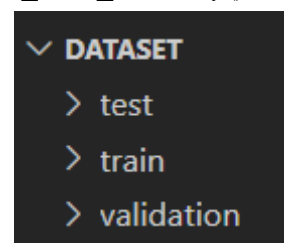
3. Descrição do modelo e implementação dos algoritmos

3.1. Pré-processamento e Normalização das imagens

Antes do treino, todas as imagens foram redimensionadas para 128×128 píxeis e convertidas para tensores numéricos com valores normalizados entre 0 e 1, de forma a garantir uma escala uniforme nos dados de entrada. Foi também aplicado o embaralhamento (shuffle) no conjunto de treino para melhorar a generalização do modelo.

O dataset foi lido e preparado com a função `image_dataset_from_directory()` da biblioteca TensorFlow/Keras, que automaticamente associa as imagens às respectivas classes a partir da estrutura de diretórios:

Cada pasta contém as 5 classes: *convertible*, *coupe*, *hatchback*, *sedan*, *suv*.



As imagens são por tanto carregadas a partir da estrutura de pastas (**train/**, **validation/**, **test/**) usando uma função auxiliar `ds()` que utiliza o `image_dataset_from_directory()`. Durante o pipeline, os píxeis das imagens são convertidos para float32 e normalizados para o intervalo [0,1]. O `prefetch(AUTOTUNE)` é usado para acelerar o treino, permitindo que as imagens sejam carregadas enquanto o modelo está a treinar.

3.2. Modelo Base - Arquitetura CNN

A Rede Neuronal Convolutacional (CNN) foi o modelo principal escolhido, devido à sua capacidade de extrair automaticamente padrões visuais e reconhecer formas e texturas nas imagens.

O modelo foi desenvolvido em Keras (TensorFlow) e tem uma estrutura sequencial simples:

- **Entrada (Input):** (128 x 128 x 3) – 128 por 128 pixels, 3 canais RGB;
- **Conv2D** - camada convolutacional com 32 filtros, cada um de tamanho 3×3 pixels. Função de ativação ReLU → extrai padrões locais iniciais;
- **MaxPooling2D 4×4** - reduz a dimensionalidade espacial da imagem (altura e largura) ao selecionar o valor máximo dentro de cada bloco 4×4 pixels. Isto mantém as características mais relevantes, reduz o número de parâmetros e ajuda a prevenir overfitting.
- **Conv2D** - camada convolutacional com 64 filtros, 3×3 pixels, função de ativação ReLU → capta padrões intermédios;
- **MaxPooling2D 4×4** – aplica novamente a redução espacial, simplificando a representação e tornando o modelo mais eficiente.
- **Conv2D** - camada convolutacional com 128 filtros, 3×3 pixels, função de ativação ReLU → aprende padrões de maior nível (formas/contornos característicos).
- **GlobalAveragePooling2D** – Melhora a generalização
- **Dropout (0.5):** Desativa aleatoriamente 50% das unidades durante o treino para regularização e diminuição de overfitting.
- **Dense (5, função de ativação Softmax)** → produz probabilidades para as 5 classes.

O modelo tem como otimizador e parâmetros de treino:

- **Otimizador:** Adam;
- **Função de perda:** *categorical_crossentropy*, pois as labels foram codificadas em formato one-hot (cada classe é representada por um vetor com 1 na posição correta e 0 nas outras, como a matriz de confusão só que com 0s e 1s);
- **Métrica:** Accuracy;
- **Épocas:** 100 (valor máximo definido, mas o treino pode parar antes devido ao callback EarlyStopping);
- **Callback (*EarlyStopping*):** monitoriza a `val_loss` (também podendo monitorizar a `val_accuracy`) e interrompe automaticamente o treino caso não haja melhoria durante 10 épocas consecutivas, restaurando os melhores pesos obtidos até ao momento. Este mecanismo evita overfitting e garante que o modelo final corresponde ao melhor desempenho obtido na fase de validação.

3.3. Modelo alternativo – Arquitetura MLP

A MLP recebe a imagem já “achatada” (sem convoluções) e aprende padrões globais através de camadas densas. É simples e rápida, mas perde informação espacial (relações entre pixels vizinhos), o que normalmente reduz o desempenho face a uma CNN.

O modelo tem também uma estrutura sequencial simples:

- **Entrada (Input):** $(128 \times 128 \times 3)$ - 128 por 128 píxeis, 3 canais RGB;
- **Flatten:** transforma a imagem numa única vetor de 49 152 características $(128 \times 128 \times 3)$;
- **Dense(32, ReLU)** → camada densa com 32 neurónios, responsável por aprender combinações lineares simples das entradas;
- **BatchNormalization()** → estabiliza a distribuição das ativações, acelera e estabiliza o treino;
- **Dense(64, ReLU)** → camada com 64 neurónios, que aprende padrões intermédios e representações mais abstratas;
- **BatchNormalization();**
- **Dense(128, ReLU)** → camada com 128 neurónios, com maior capacidade para padrões de alto nível;
- **BatchNormalization();**
- **Dropout (0.5):** Desativa aleatoriamente 50% das unidades durante o treino para regularização e diminuição de overfitting.
- **Dense (5, função de ativação Softmax)** → produz probabilidades para as 5 classes.

Neste modelo MLP optou-se por utilizar a biblioteca Keras, integrada no TensorFlow, através da classe `keras.Sequential` (tal como no modelo CNN), em vez de recorrer à biblioteca `scikit-learn` e ao seu `sklearn.neural_network.MLPClassifier`, que foi a abordagem usada nas aulas práticas.

Esta decisão deve-se a várias razões técnicas e práticas. Em primeiro lugar, o Keras permite que o modelo seja treinado utilizando a GPU, o que acelera significativamente o processo de treino, especialmente em tarefas que envolvem imagens e grandes volumes de dados. Além disso, o Keras oferece um maior controlo e flexibilidade na definição da rede neuronal, permitindo configurar manualmente todas as camadas (por exemplo, Dense, BatchNormalization, Dropout, etc).

Outra razão para esta escolha foi o facto de se pretender aproximar o modelo MLP ao máximo da arquitetura CNN desenvolvida, de modo a realizar uma comparação direta entre ambos. Este ponto vai ser abordado mais para a frente.

3.4. Configuração do Ambiente e Bibliotecas Utilizadas

O projeto foi desenvolvido em **Python 3.10.18**, utilizando as seguintes bibliotecas principais:

- **TensorFlow / Keras** – para criação, treino e avaliação dos modelos de rede neuronal (CNN e MLP).
- **NumPy** – para manipulação de vetores, matrizes e processamento de previsões.
- **Matplotlib** – para visualização de resultados e geração da matriz de confusão.
- **scikit-learn** – para cálculo de métricas estatísticas: *accuracy*, *precision*, *recall* (*sensibilidade*), *f1-score*, *AUC* e *especificidade*.
- **Pickle** – para guardar o histórico de treino (`history.pkl`).

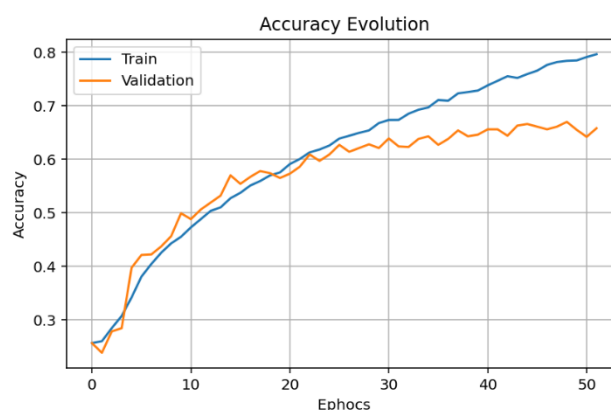
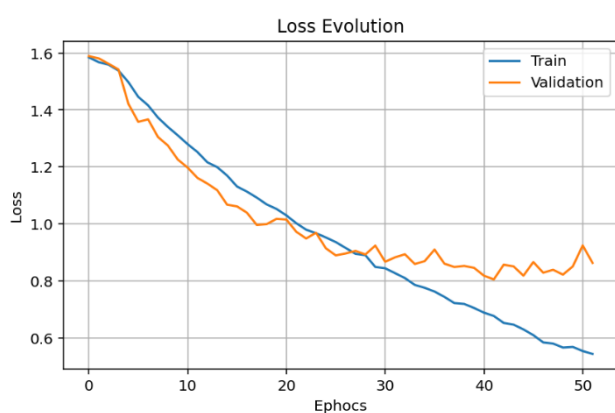
A execução foi feita em ambiente local com GPU (quando possível), garantindo tempos de treino mais rápidos.

4. Análise de resultados

4.1. Análise do Modelo Base (CNN)

Curvas de treino/validação.

Observa-se descida consistente da loss de treino e validação até ~época 25, seguida de melhoria mais lenta e estabilidade. A accuracy de validação acompanha a de treino com um gap moderado, o que indica boa generalização. O EarlyStopping interrompe por volta da época ~50 com restauração dos melhores pesos.



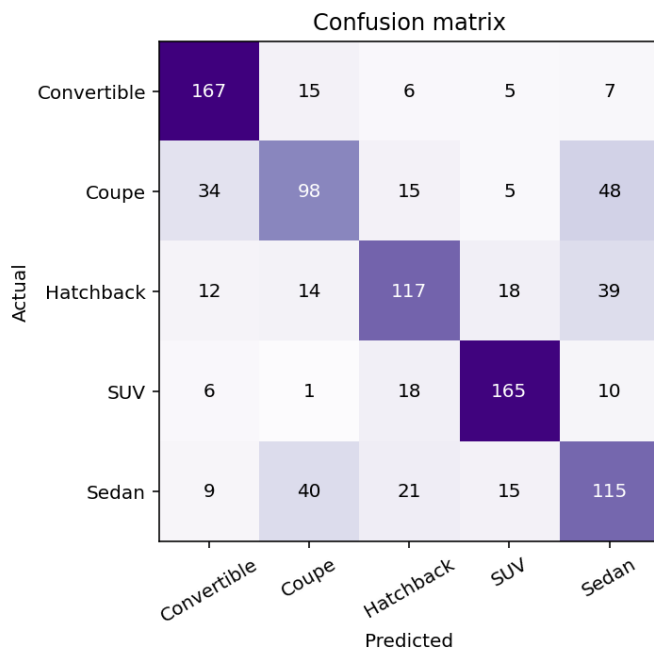
Conjunto de teste

O modelo CNN apresentou um bom desempenho geral, alcançando 66,2 % de accuracy no conjunto de teste e um AUC macro de 0,8967, o que demonstra uma boa capacidade de separação entre as diferentes classes. O desempenho foi analisado através das principais métricas pedidas: accuracy, sensibilidade (recall), especificidade, F1-score (f-measure) e AUC, permitindo uma avaliação completa do classificador.

A partir da matriz de confusão, observa-se que as classes SUV e Convertible foram as mais corretamente classificadas, com 165/200 e 167/200 acertos, respetivamente. Estas categorias obtiveram também os valores mais elevados de F1-score ($\approx 0,80$) e de especificidade ($> 0,92$), indicando que o modelo distingue bem veículos destas categorias.

Por outro lado, as classes Coupe e Hatchback continuam a apresentar maior confusão, sobretudo entre si e com “Sedan”. A precisão mais baixa nestas classes (≈ 0.58 – 0.66) mostra que ainda há sobreposição visual relevante, causada por formatos semelhantes e ângulos de captação.

A classe Sedan, apesar de ser ligeiramente menos precisa, apresenta um recall equilibrado (0.575), sugerindo que o modelo reconhece a maioria dos exemplos, mas ainda tende a confundi-la com “Coupe” e “Hatchback”.



```
Found 1000 files belonging to 5 classes.
Accuracy: 0.662
Relatório (precision, recall(sensibilidade), F1):
      precision    recall  f1-score   support

Convertible    0.7325    0.8350    0.7804     200
Coupe          0.5833    0.4900    0.5326     200
Hatchback      0.6610    0.5850    0.6207     200
SUV            0.7933    0.8250    0.8088     200
Sedan          0.5251    0.5750    0.5489     200

 accuracy
macro avg      0.6590    0.6620    0.6583    1000
weighted avg   0.6590    0.6620    0.6583    1000

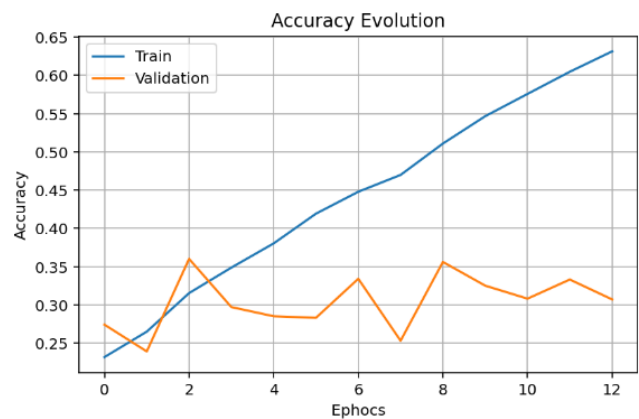
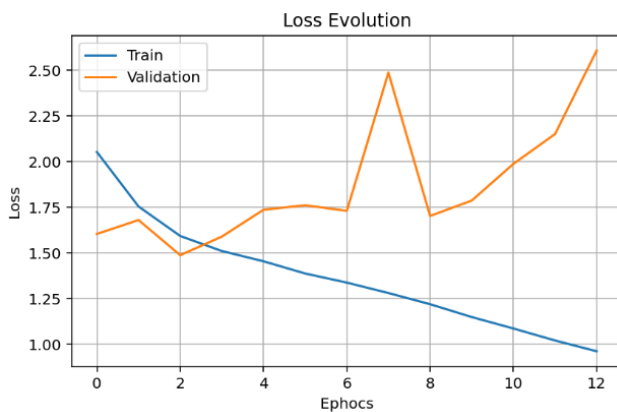
Especificidade por classe:
Convertible: 0.9237
Coupe: 0.9125
Hatchback: 0.9250
SUV: 0.9463
Sedan: 0.8700

Macro AUC (OvR): 0.8967
```

4.2. Análise do Modelo Alternativo (MLP)

Curvas de treino/validação

Observa-se uma redução contínua da loss no conjunto de treino, enquanto a loss de validação apresenta oscilações e tendência de aumento, evidenciando overfitting a partir das primeiras épocas. A accuracy de treino cresce de forma consistente, mas a accuracy de validação mantém-se estável e consideravelmente mais baixa, mostrando que o modelo tem dificuldade em generalizar para dados não vistos.

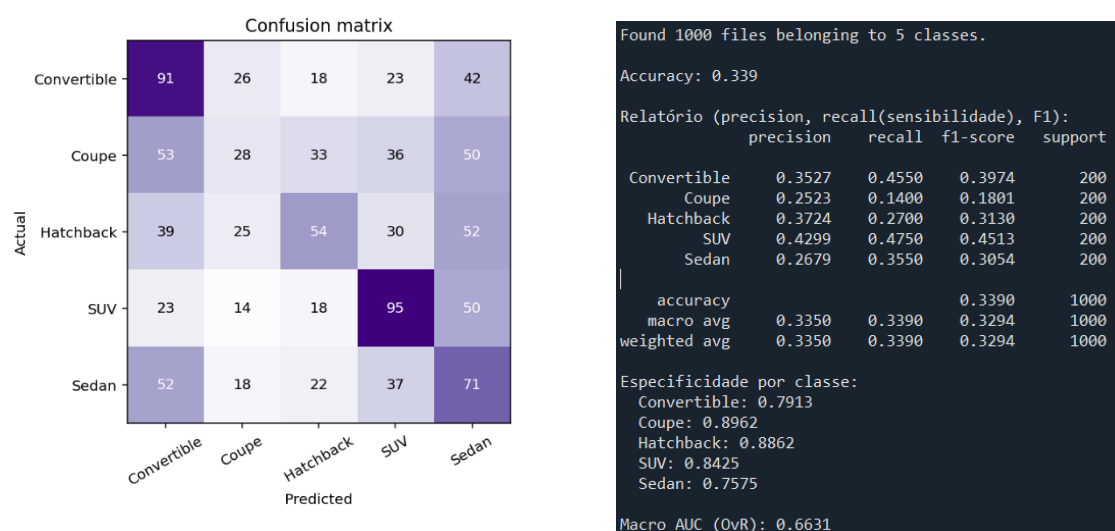


Conjunto de teste

O modelo MLP obteve um desempenho inferior ao da CNN, alcançando 33,9 % de accuracy e um AUC macro de 0,6631.

A matriz de confusão revela um número elevado de erros e confusões entre quase todas as classes, especialmente entre Sedan, Convertible e Hatchback.

As classes apresentam valores de F1-score bastante baixos, confirmando que a rede não consegue distinguir bem os padrões visuais complexos presentes nas imagens.



4.3. Comparação entre MLP e CNN

Os resultados práticos são claros: a CNN atingiu 66,2 % de accuracy e um AUC de 0,8967, enquanto a MLP obteve apenas 33,9 % de accuracy e AUC de 0,6631. Para além dos números, existem razões estruturais que explicam por que motivo a CNN é a escolha mais adequada para problemas de imagem.

A rede neuronal MLP começa por achatar a imagem ($128 \times 128 \times 3 \rightarrow 49\,152$ valores) e, ao fazê-lo, perde a informação espacial, isto é, a relação entre píxeis vizinhos. Na prática, trata cada píxel como uma unidade independente.

A CNN, por outro lado, opera com filtros bidimensionais (2D) que exploram vizinhanças locais (receptive fields), permitindo captar arestas, curvas e padrões de forma consistente no espaço. Esta capacidade de preservar a estrutura espacial é fundamental na classificação de imagens de carros, onde a forma e a disposição dos elementos visuais são determinantes.

5. Conclusões

Nesta primeira meta do projeto, foi desenvolvido um sistema de classificação automática de imagens de veículos em cinco categorias de carroçaria (Convertible, Coupé, Hatchback, Sedan e SUV), recorrendo a redes neuronais.

Durante o desenvolvimento, a principal dificuldade encontrada foi o controlo do overfitting nas primeiras versões do modelo.

Inicialmente, a rede apresentava uma elevada accuracy no treino, mas um desempenho significativamente inferior na validação, o que indicava uma falta de generalização. A introdução de técnicas como Dropout, EarlyStopping e GlobalAveragePooling permitiu mitigar esse problema, estabilizando o treino e garantindo resultados mais consistentes e equilibrados entre os diferentes conjuntos de dados.

Foram implementados e avaliados dois modelos distintos: MLP (Multilayer Perceptron) e CNN (Convolutional Neural Network), utilizando o mesmo conjunto de dados e condições experimentais, com o objetivo de comparar a sua eficácia.

Os resultados mostraram que o modelo CNN atingiu um desempenho significativamente superior, com 66,2% de accuracy e AUC macro de 0,8967, enquanto o MLP obteve 33,9% de accuracy e AUC de 0,6631.

O modelo CNN demonstrou boa estabilidade e generalização, com curvas de treino e validação consistentes e sem sinais de sobreajuste significativo. A análise das métricas confirmou a eficácia da arquitetura implementada, validando a escolha da CNN como modelo base para continuação do trabalho nas próximas metas.

Na Meta II, o foco será otimizar o desempenho da rede, explorando novas estratégias.

Assim, a Meta I cumpriu integralmente os objetivos propostos, resultando num modelo funcional, analisado e devidamente validado.

6. Referências

https://www.w3schools.com/python/python_ml_getting_started.asp

<https://keras.io/api/models/>

https://scikit-learn.org/stable/getting_started.html

<https://www.geeksforgeeks.org/deep-learning/convolutional-neural-network-cnn-in-machine-learning/>

<https://www.kaggle.com/datasets/ademboukhris/cars-body-type-cropped/data>

<https://www.kaggle.com/datasets/mayurmahurkar/stanford-car-body-type-data>